tree link by link, and check the string key character by character, until the condition at line #2 of listing 6.2 returns `true` and we go to line #3.

The result of line #3 is the only difference between successful and unsuccessful search: looking at the example of figure 6.7, a successful search for the word "`ant`" would follow the exact same steps, with the only difference that the final node (denoted with `T` in the rightmost diagram) would have to be a key node.

Notice that we can avoid checking for empty tries or handling the root as a special case, in listing 6.3, because in the trie constructor, in listing 6.1, we create the root as an empty node; this (together with careful implementation of all the methods) will also support our assumption that the `node` argument in `search` (and all the other methods) is never `null`.

Listing 6.4 method searchNode

```
function searchNode(node, s)
  if s == "" then
      return node
  c, tail ← s.splitAt(0)
  if node.children[c] == null then
    return null
  else
    return search(node.children[c], tail)
```

The search method is the most important method for tries, because all the other methods will be based on it; search is so crucial to those implementations that we provide, in